

16. Design of Convolutional layer using RTL

AIM

To design and simulate a **3×3 Convolutional Layer** using RTL (Verilog HDL) for image processing and convolutional neural network applications.

SOFTWARE REQUIRED

- Xilinx Vivado Design Suite
- Vivado Simulator (XSIM)
- Verilog HDL
- Windows / Linux Operating System

PROCEDURE

1. Open **Xilinx Vivado** and create a new RTL project.
2. Select the target FPGA device or board.
3. Create a Verilog module to implement a **3×3 convolution operation**.
4. Provide nine 8-bit input pixels corresponding to a 3×3 input window.
5. Assign nine 8-bit kernel coefficients representing the convolution filter.
6. Implement **multiply–accumulate (MAC)** operations between the input pixels and kernel values.
7. Add a single bias value to the accumulated sum.
8. Write a testbench to apply different pixel values, kernel coefficients, and bias.
9. Simulate the design using **Vivado Simulator**.
10. Observe the output waveform and verify the convolution result with manual calculations.

RTL program for 3x3 convolution

```
module conv2d_3x3 (  
    input signed [7:0] p0, p1, p2,  
        p3, p4, p5,  
        p6, p7, p8,    // 3×3 input window  
    input signed [7:0] k0, k1, k2,  
        k3, k4, k5,  
        k6, k7, k8,    // 3×3 kernel  
  
    input signed [15:0] bias,  
    output signed [19:0] y        // convolution output
```

```
);
```

```
wire signed [15:0] m0, m1, m2, m3, m4, m5, m6, m7, m8;  
wire signed [19:0] sum;  
assign m0 = p0 * k0;  
assign m1 = p1 * k1;  
assign m2 = p2 * k2;  
assign m3 = p3 * k3;  
assign m4 = p4 * k4;  
assign m5 = p5 * k5;  
assign m6 = p6 * k6;  
assign m7 = p7 * k7;  
assign m8 = p8 * k8;  
assign sum = m0 + m1 + m2 + m3 + m4 + m5 + m6 + m7 + m8;  
assign y = sum + bias;  
endmodule
```

Testbench program for 3x3 convolution

```
`timescale 1ns / 1ps  
module tb_conv2d_3x3;  
    // Inputs  
    reg signed [7:0] p0, p1, p2,  
        p3, p4, p5,  
        p6, p7, p8;  
    reg signed [7:0] k0, k1, k2,  
        k3, k4, k5,  
        k6, k7, k8;  
    reg signed [15:0] bias;  
    // Output  
    wire signed [19:0] y;  
    // Instantiate DUT  
    conv2d_3x3 uut (
```

```

.p0(p0), .p1(p1), .p2(p2),
.p3(p3), .p4(p4), .p5(p5),
.p6(p6), .p7(p7), .p8(p8),
.k0(k0), .k1(k1), .k2(k2),
.k3(k3), .k4(k4), .k5(k5),
.k6(k6), .k7(k7), .k8(k8),
.bias(bias),
.y(y)
);

```

initial begin

```

// Test Case 1: All ones kernel
p0=8'd10; p1=8'd10; p2=8'd10;
p3=8'd10; p4=8'd10; p5=8'd10;
p6=8'd10; p7=8'd10; p8=8'd10;
k0=8'd1; k1=8'd1; k2=8'd1;
k3=8'd1; k4=8'd1; k5=8'd1;
k6=8'd1; k7=8'd1; k8=8'd1;
bias = 16'd5;

#10;

// Test Case 2: Edge detection-like kernel
p0=8'd5; p1=8'd5; p2=8'd5;
p3=8'd5; p4=8'd20; p5=8'd5;
p6=8'd5; p7=8'd5; p8=8'd5;
k0=-8'd1; k1=-8'd1; k2=-8'd1;
k3=-8'd1; k4= 8'd8; k5=-8'd1;
k6=-8'd1; k7=-8'd1; k8=-8'd1;
bias = 16'd0;

#10;

// Test Case 3: Mixed signed values
p0=8'd10; p1=-8'd5; p2=8'd3;
p3=8'd2; p4=8'd1; p5=-8'd4;

```

```

p6=8'd6; p7=8'd7; p8=8'd8;

k0=8'd1; k1=8'd2; k2=8'd1;

k3=8'd0; k4=8'd1; k5=8'd0;

k6=8'd1; k7=8'd2; k8=8'd1;

bias = 16'd2;

#10;

$stop;

end

endmodule

```

Output

> p0[7:0]	10	10	5	10
> p1[7:0]	-5	10	5	-5
> p2[7:0]	3	10	5	3
> p3[7:0]	2	10	5	2
> p4[7:0]	1	10	20	1
> p5[7:0]	-4	10	5	-4
> p6[7:0]	6	10	5	6
> p7[7:0]	7	10	5	7
> p8[7:0]	8	10	5	8
> k0[7:0]	1	1	-1	1
> k1[7:0]	2	1	-1	2
> k2[7:0]	1	1	-1	1
> k3[7:0]	0	1	-1	0
> k4[7:0]	1	1	8	1
> k5[7:0]	0	1	-1	0
> k6[7:0]	1	1	-1	1
> k7[7:0]	2	1	-1	2
> k8[7:0]	1	1	-1	1
> bias[15:0]	2	5	0	2
> y[19:0]	34	95	120	34

RESULT

The 3×3 Convolutional Layer was successfully designed and simulated using RTL (Verilog HDL). The module correctly performed multiplication of input pixels with kernel coefficients, accumulated the results, and added the bias to produce the convolution output. The simulation results matched the expected theoretical values, confirming the correct functionality of the convolutional layer.